

```
[root@localhost raju]# chef-client
Starting Chef Client, version 12.6.0
resolving Cookbooks for run list: ["cookbook_101"]
Synchronizing Cookbooks:
- cookbook_101 (0.1.0)
Compiling Cookbooks...
Converging 11 resources
Recipe: cookbook_101::default
  * file[/home/raju/regular_file] action create
    - create new file /home/raju/regular_file
    - update content in file /home/raju/regular_file from none to 8b41c2
    - change owner from '' to 'raju'
    - restore selinux security context
  * directory[/home/raju/dir_file] action create
    - create new directory /home/raju/dir_file
    - change mode from '' to '0744'
    - change owner from '' to 'root'
    - change group from '' to 'root'
    - restore selinux security context
  * link[/home/raju/symlink_list.txt] action create
    - create symlink at /home/raju/symlink_list.txt to /home/raju/list.txt
  * execute[my_command] action run
    - execute ls -l /home/raju/lab_demo | grep ^d | wc -l > /home/raju/totdir
  * script[execute_given_script] action run
    - execute "bash" /tmp/chef-script20151228-19807-lriawvv
  * file[/home/raju/testcp.txt] action create
    - create new file /home/raju/testcp.txt
    - update content in file /home/raju/testcp.txt from none to 7a98d4
    - restore selinux security context
  * bash[take_backup] action run
    - execute "bash" /tmp/chef-script20151228-19807-ycupdc
  * service[iprdump] action stop
    - stop service service[iprdump]
  * ifconfig[192.168.122.55] action add
    - run ifconfig eth0 192.168.122.55 to add ifconfig[192.168.122.55]
Recipe: <Dynamically Defined Resource>
  * file[/etc/sysconfig/network-scripts/ifcfg-eth0] action create (up to date)
Recipe: cookbook_101::default
  * user[newuser] action create
    - create user newuser
  * cron[chef-client] action create
    - add cronbat entry for cron[chef-client]
Running handlers:
Running handlers complete
Chef Client finished, 11/12 resources updated in 07 seconds
```

Figure 4: Chef-client execution output

in Figure 3.8 shows how to stop the *iprdump* service.

We can set the static IP address of a network device *eth0* in a different way. The code given in Figure 3.9 explains how to set up the IP address with *ifconfig* resource on *eth0* in a cookbook.

One of the main tasks for a sys admin is user management — creating users, updating users, deleting user accounts and settings, locking or unlocking user passwords, etc. The code block in Figure 3.10 explains how the user resource type is used to create a user name *newuser* and the user's home directory */home/newuser*; */bin/bash* is the login shell.

If you want to automate particular tasks, you can use the cron job to do this periodically within a desired time frame. Once we add run lists to nodes from Chef-server, we have to execute Chef-client commands to all the nodes to execute the run lists. If you have a large number of nodes, then frequently updating the cookbook will make it a tedious task to execute Chef-client in all the nodes after every cookbook update. To automate this, write a cron job as shown in Figure 3.11. Cron is the resource type, and Chef-client is the name of the resource block. In general, using cron, we schedule routine background tasks at a given time interval on an ongoing

basis. The time interval is specified in minutes, hours, day of the week and the month in which the command is to be executed. Here we expect, on an average, to upload the cookbook into nodes every 30 minutes; so we set 'minute' to 30 and execute the command as the root user.

The output of the Chef-client command execution is stored in the *Chef_client_log* file. Every execution of Chef-client will append the log file with its output due to >> being used in the command.

Create a cookbook file as *cookbook_101* using the *Skife cookbook create cookbook_101* command, go to /*Chef-repo/cookbooks/cookbook_101/recipe/* directory, copy the given code examples (Figures 3.1 to 3.11) into the *default.rb* file and then save it.

To add the run list, specified in *cookbook_101*, execute the following command in the server: # *knife node run_list add demonode "recipe[cookbook_101]"*.

Once the run list specified in the *cookbook_101* is added into the demonode successfully, we need to execute the Chef-client command in the Chef-client system to execute this list. The Chef-client downloads

the required system configuration in the form of a run list from the Chef-server and configures the node as mandated in *cookbook_101*.

The screenshot of the Chef-client command execution output is shown in Figure 4. From the figure, we can interpret that 11 resources are specified as recipes in the *cookbook_101*. All these resources are configured in the demonode and set the desired state.

In the next article, we will dive deeper inside the cookbook and write advanced level recipes. For example, Linux flavours maintain different package names and services to set up a server. It is not advisable to write a specific cookbook for every Linux flavour (RHEL, CentOS, Fedora, Ubuntu etc.); instead we can write a cookbook in such a way that it will work for any Linux distribution. 

By: Dr B. Thangaraju and Magesh Kasthuri

Thangaraju is an open source software (OSS) evangelist, who works at talent transformation, Wipro Technologies, Bengaluru. He can be reached at balat.raju@wipro.com.

Kasthuri is a senior distinguished member of the technical staff in Wipro. He is a senior technical consultant in the Java-Technology Practices Group and can be reached at magesh.kasthuri@wipro.com.